

DWI Prompt Optimizer

Management Brief, Current Evidence, and Future Direction

Plain-language purpose: the value demonstrated today, the next evidence to collect, and how DWI could later support responsible AI adoption

Evidence date: 1 September 2026

Audience: business, engineering, delivery, operations, and AI leadership

Product evidence version: 5088dbf6e5fdeea2415b991c429de8e5229d7a7b

Status: working pre-release proof of concept; the core local workflow is demonstrated, and Phase 4 is deliberately frozen after a clear decision-governance failure

Important boundary. This document does not claim production readiness, release approval, project-management integration, a deployed management reporting system, or measured live-AI uplift. It also does not soften the Phase 4 governance failure described below.

Executive summary

DWI helps developers give AI the right project background before asking it to do work. After login, it gathers a limited set of project facts with the developer's consent, shows those facts for review and correction, and saves only the approved project summary. Prompt Optimizer then reuses that approved knowledge for later tasks. It is more than a generic prompt editor.

The current working proof of concept provides a consistent journey: review approved project facts, describe the work, see sources and assumptions, identify important missing information, prepare a local draft, review it, save it, or reset the current task without deleting approved project knowledge. Engineering checks passed on the recorded implementation version, including 421 tests, type checks, packaging, installed-flow checks, narrow-screen review, and GitHub-hosted continuous integration. This means the core code, build, and package evidence is not limited to one laptop.

Three evidence boundaries guide the next controlled steps:

- DWI can show an important unanswered question, but the guided one-question-at-a-time answer step is not finished.
- The packaged extension has passed a recorded restart, resume, reset, and project-context preservation journey. The latest repeat was blocked by macOS automation before product checks began, so it neither replaces the earlier pass nor records a product failure.
- Phase 4 exposed a clear decision-governance failure. The human approver approved maximum calls, no retries, and monetary ceilings. During contract preparation, 20,000 aggregate and 12,000 per-call token ceilings were added without explicit approval from the human approver. The 12,000 ceiling then became pivotal execution logic and stopped the evaluation.

The stop control did what it was coded to do: it halted after the 14,531-token readiness response and preserved the evidence. The decision process failed earlier. A number must not gain business authority merely because it appears in code or configuration—even when produced by a multiplier. Any rule that can change execution, cost, data sharing, or a phase outcome must trace to an approved requirement, named human approver, explainable calculation, version, test, and rollback.

Phase 4 remains deliberately frozen. The human approver did not authorize a retry or ceiling increase, and the AI assistant did not perform either action, so this failure could not be tuned into a convenient pass. Before development resumes, DWI needs an audit of every pivotal limit, default, multiplier, exception, and fallback that may have entered silently. No prompt-quality conclusion can be drawn because the comparison never began; quality remains unmeasured.

The future opportunity is to connect approved task information from a project-management system to DWI. Agreed rules could turn task priority, risk, complexity, approved project-context size, sprint capacity, and fixed service overhead into a clear, adaptive AI usage allowance. DWI's internal prompt-building engine could use that allowance when deciding how much approved context to include. Separately, outcome-only feedback—such as helpful or not helpful, corrected, saved, reset, time taken, and allowance adherence—could be grouped for management insight without copying prompts, generated output, code, project content, file paths, or employee names.

These future features are not built today. An adaptive usage allowance would never authorize spending or external AI use, and feedback would improve tools, training, and delivery practices—not rank or monitor individual employees.

1. The management challenge

AI use can spread quickly while different delivery groups develop their own habits. Without a shared approach, the organization may see more activity without knowing whether work is becoming faster, clearer, safer, or easier to review.

Four common problems

Problem	What it means in everyday work	Why leaders should care
Repeated setup	Developers explain the same project background again for each task and tool	Time is lost and the same facts are described differently
Uneven results	Important rules, risks, and assumptions may be missing	Polished-looking work can still create review effort or rework
Unclear AI effort	Priority and complexity are not linked to an agreed usage allowance	AI capacity and cost are harder to plan and explain
Little useful feedback	Usage counts show activity, not whether AI actually helped	Leaders cannot target training, tools, or process improvements confidently

The opportunity

DWI offers one clear way of working from approved project knowledge to a reviewed result. The aim is not to remove human judgment. The aim is to give developers a better starting point, make assumptions visible, and keep decisions with the developer.

This creates a possible foundation for responsible growth later: project priority can inform a clear limit, work remains reviewable, and privacy-safe outcome information can help improve the system.

2. What DWI does today

Start with project knowledge the developer approves

DWI begins after login with a limited project check. The developer is told what will be inspected and gives consent. DWI prepares a short project summary, and the developer can review, correct, and approve it.

That approved project summary is saved for later work. In the product it is called **Project Meta Context**. In plain terms, it is the agreed set of project facts that later tasks may reuse.

The current journey

Step	What the developer does	What DWI contributes
1. Review project facts	Checks and approves the project summary	Reuses bounded project knowledge instead of starting from nothing
2. Describe the task	Enters the task, expected result, and importance	Gives each AI-assisted task a consistent starting form
3. See what is known	Reviews sources, assumptions, and missing information	Shows where information came from and where an answer is still needed
4. Prepare a local draft	Moves through the local preparation steps	Produces a rule-based draft without requiring an external AI service
5. Review and decide	Corrects, saves, copies, or rejects the work	Keeps the final decision with the developer
6. Reset one task	Clears the current task when needed	Keeps the approved project facts for the next task

What is demonstrated today and what comes next

Working capability demonstrated today	Separate next-stage capability or evidence
Approved project summary	Production-wide rollout
Clear task and importance fields	Finished guided question-and-answer step
Visible sources and assumptions	Phase 4 decision-governance correction, followed later by a controlled live-AI comparison
Important missing information shown	Project-management system connection
Local rule-based draft and review	Adaptive task-, project-, and sprint-informed AI allowances
Saved work and safe task reset	Management dashboards or group reporting
Exact-version GitHub-hosted tests, build, and packaging	Independent operational pilot and production-scale evidence

3. Benefits for people and delivery

Benefits visible in the working proof of concept

- **Less setup time:** approved project knowledge can be reused instead of explained again.
- **Fewer missed requirements:** known rules and constraints stay visible while work is prepared.
- **Less mental effort:** developers spend less time searching, repeating, and tracking information across tools.
- **Easier review:** sources, assumptions, and decisions are simpler to check.
- **Shared understanding:** delivery groups can begin from the same approved project facts.
- **Developer control:** the developer reviews, corrects, saves, or resets the work.

Value at each level

Level	Main benefit	Decision supported
Developer	Clearer day-to-day work with less repeated setup	Which support helps this task
Delivery lead	More consistent handoffs and fewer recurring knowledge gaps	Which shared knowledge or working practice needs improvement
Software delivery operations	Better view of delays, fallback use, and process friction	Which delivery or platform problem to fix first
Management and AI Center of Excellence	Better view of useful adoption, training needs, and overall capacity	Where to invest, expand, constrain, or stop

Why this is different from measuring usage alone

More prompts or more AI activity do not automatically mean more value. High activity may indicate repeated attempts, confusion, or poor results. Management should look at usefulness, correction effort, time to a usable draft, safe fallback, and adherence to agreed limits—not just volume.

4. Evidence boundaries and next controlled steps

4.1 Complete the guided question step

DWI can identify important missing information, but it cannot yet complete the short answer-or-skip journey safely. Finish this step before future project-management data can influence how much context DWI uses.

4.2 Refresh restart evidence on the exact version

The packaged extension has passed the recorded cross-process restart, resume, reset, and project-context preservation journey in an isolated profile. A later repeat stopped before product checks because macOS automation could not see the consent window. That does not erase the earlier evidence; independent repetition on the exact current version in a second environment remains useful.

4.3 Phase 4: governance failure found before quality measurement

The human approver approved the provider route and synthetic inputs, maximum calls, no retries, and monetary ceilings. During contract preparation, the AI assistant added 20,000 aggregate, 120,000 evaluation, and 12,000 per-completion token ceilings without explicit approval from the human approver. The program record then described the combined bounds as user-supplied. That attribution is not supported by the retained approval message.

The 120-byte readiness request reached the configured route and returned HTTP 200. The service reported 14,526 input plus 5 output tokens, or 14,531 total. That was below the 20,000 aggregate ceiling added by the AI assistant but above the unapproved 12,000 per-call ceiling, so the budget control stopped the run before any business task, optimized output, or quality comparison. The agreed no-retry rule then prevented another attempt.

The token check happened after the response, so it prevented later calls but could not prevent the first call's usage. The approved dollar ceilings were documented, but validated pricing and billing data were unavailable. They therefore did not operate as a true pre-call monetary control, and actual cost remains unknown.

Fail-closed enforcement worked after the response; policy authority failed before it. This is a clear control-system failure. It is not evidence that prompt quality was poor, because prompt quality was never tested.

The failure remains intentionally visible. Phase 4 stays deferred with no retry and no ceiling change while DWI audits every decision-bearing limit, default, multiplier, exception, routing choice, data rule, fallback, scoring threshold, and phase gate. Each must have a business source, named human approver, explicit approval, explainable derivation, version, test, and rollback. Missing authority must stop execution before an external call.

4.4 Production remains a later decision

The working proof of concept is published on the functional branch, and exact-commit GitHub CI has independently repeated the repository checks. Production promotion remains a separate decision requiring the controlled provider comparison, pilot evidence, and explicit merge and release approval.

5. Future direction: project-management data, adaptive AI allowances, and safe feedback

These ideas describe a possible next stage. They are not built into the current product.

5.1 Optional project-management connection

A future connection to a project-management system could provide a small approved set of task details:

- task reference, type of work, and delivery stage;
- delivery period or sprint, timing, and priority;
- risk, broad complexity band, and version of the approved company rule.

DWI should still work when this connection is unavailable. The project-management system should not become a required dependency for the basic local journey.

5.2 Approval-led, adaptive AI usage allowance

Agreed company rules could turn approved task details into a clear usage allowance. This is sometimes called a **token budget**, because tokens are the units used to measure AI input and output.

The first requirement is authority, not adaptiveness. A declarative file may record an approved decision; it must not create one. Every input, default, multiplier, formula, clamp, exception, and hard limit must trace to an approved business requirement. An imperative decision layer must validate that authority, reject missing or conflicting rules before a provider call, calculate the allowance, explain it, and reconcile planned use with observed use.

The allowance could then separate fixed service or route overhead from work-dependent needs such as approved project-context size, task scope, complexity, criticality, expected output, and reserve. It could also respect the remaining sprint or delivery-group allocation while staying inside independently approved hard maximums.

The allowance could help DWI decide how much approved project context to include and how deeply to prepare the task. The order of control should remain simple:

Explicit rule approval → consent and privacy → hard safety limits → imperative policy check → adaptive task allowance → explicit external-AI action → developer review

The usage allowance must never:

- approve spending;
- approve external AI use;
- originate from an unapproved default or multiplier;
- raise a hard safety limit;
- send information without a clear user action;
- replace developer review.

5.3 Outcome-only feedback

Future feedback could record simple outcomes without copying the work itself:

- helpful, mixed, or not helpful;
- saved, reused, corrected, reset, or abandoned;
- type of correction, without the text;
- missing question answered or skipped;
- local fallback used;
- broad time band;
- estimated and observed AI usage where available;
- stayed within the agreed limit or stopped safely.

This feedback could help improve guidance, training, budgets, and delivery processes. It must not copy prompts, generated results, code, project content, file paths, free-text notes, or employee names.

5.4 The future improvement loop

Approved task details → agreed company rule → adaptive AI allowance → DWI prepares work → developer decision → outcome-only feedback → group-level improvement

The loop should advise people. It should not automatically change safety rules or score an employee.

6. Management goals and useful measures

Management goal	Useful information	Decision it supports
Useful AI adoption	Helpful outcomes, save or reuse, correction type, workflow coverage	Where AI helps and where training or redesign is needed
Predictable AI capacity and cost	Planned allowance, observed use, adherence, time band	Capacity planning, budget bands, and exception rules
Better delivery quality	Missing-information rate, correction patterns, rework, fallback	Which workflow stages need better context or checks
More consistent knowledge	Reuse of approved project facts, recurring missing-information categories	Which project knowledge or guidance needs improvement
Responsible control	Consent state, rule version, safe-stop reason, explicit action	Whether safeguards are working as intended
Better developer experience	Usefulness, correction burden, reset or abandon outcome	Tool, training, and support priorities

Recommended measures for a small pilot

- use of approved project knowledge;
- time to a developer-usable first draft;
- important missing information found and resolved;
- accepted, corrected, abandoned, or reset outcome;
- avoidable rework category;
- fallback and failure rate;
- difference between estimated and observed AI usage;
- percentage of work staying within the agreed limit;
- usefulness by workflow or group;
- opt-out and deletion requests handled correctly.

Each measure should answer a real decision. If no decision would change, the organization should not collect the data.

7. Privacy, employee trust, and spending safeguards

Non-negotiable rules

- DWI continues to work without a project-management or reporting service.
- The developer approves project facts before they are reused.
- Moving through the local steps does not call an external AI service.
- An adaptive AI allowance is a cap, not permission to spend or transmit information.
- Hard privacy and safety limits always take priority over task urgency.
- The developer reviews the result and decides what to keep.
- Raw prompts, generated results, code, project files, paths, and free-text notes stay out of management reporting.
- Reports are grouped by workflow or a sufficiently large group, not by named person.
- Individual ranking, performance scoring, and disciplinary use are prohibited.
- People can understand the data collected, its purpose, retention, access, and deletion rules.

Information handling in plain language

Information	Example	Default handling
Approved project facts	Reviewed project summary	Kept with the local project; not copied into management reporting
Task content	Prompt, source text, generated draft	Kept local to the task; excluded from reusable reporting
Optional written feedback	Developer's free-text note	Local unless the developer separately chooses to share it
Outcome-only feedback	Helpful, saved, corrected, time band	May be grouped only under a reviewed privacy agreement
Project-management details	Priority, risk, delivery stage	Keep to the minimum fields needed for the agreed rule
Company rule	Version, limit, reason	Recorded so the decision can be explained and rolled back

Employee-trust rule

The feedback loop exists to improve the product, training, project knowledge, and software delivery. It is not a measure of individual developer productivity.

8. Recommended path from demo to small pilot

Stage 1 — Try the current local demo

- use representative developer work;
- look for clearer tasks, reused project facts, fewer omissions, and easier review;
- collect local qualitative feedback without centralizing raw content;
- decide whether the idea is useful enough to continue.

Stage 2 — Finish the core experience

- complete the guided one-question answer or skip step;
- ensure an old answer cannot affect a changed task;
- repeat the restart, resume, and reset journey on the same version;
- record the exact product version and result.

Stage 3 — Audit and approve the decision system

- inventory every existing rule that can affect execution, cost, data sharing, quality scoring, or a phase outcome;
- identify silent defaults, multipliers, exceptions, and records that overstate approval;
- give each pivotal rule a source, named human approver, explicit approval, version, test, and rollback;
- design an imperative decision layer that rejects missing authority before execution;
- agree what outcome information may be collected;
- choose success and stop measures before the pilot.

Stage 4 — Run a dry-run

- calculate suggested limits without changing real work;
- prove that an unapproved number cannot become executable policy;
- test missing or old project-management data;
- test urgent tasks that conflict with a hard limit;
- confirm the dry-run cannot approve spending or call an external AI service;
- check that every suggested limit is understandable.

Stage 5 — Consider one small controlled pilot

- use one workflow, one group, one rule version, one time period, and one agreed maximum;
- keep external AI use explicit and human review mandatory;
- stop on a privacy, safety, or spending-rule breach;
- decide whether to improve, expand, constrain, or stop.

9. Management decision

Decision requested now

Experience the current local proof of concept and decide whether it is useful enough to justify completing the core experience and designing a small dry-run.

Decisions not requested now

- production rollout;
- release or marketplace publication;
- resuming Phase 4 development or changing its preserved evidence;
- any new live external-AI comparison before a revised accounting contract and separate approval;
- permission to spend or transmit project information;
- project-management system integration;
- management reporting deployment;
- employee monitoring;
- merge to the main branch or product baseline approval.

Questions for the meeting

1. Does starting from approved project knowledge solve a meaningful delivery problem?
2. Which pivotal decisions require a named human approver and explicit approval?
3. Which one workflow would provide the clearest, lowest-risk demonstration?
4. Which outcomes would prove usefulness beyond simple activity counts?
5. Who is the human approver for the rule, dry-run authorization, and stop decision?

Recommended management question

Is the current workflow useful enough to justify a small, controlled dry-run of the future project-management, AI-limit, and feedback model?

Appendix A — Engineering evidence and exact version

This appendix records the main evidence without changing the plain-language management claim.

Evidence item	Recorded result
Product evidence version	5088dbf6e5fdeea2415b991c429de8e5229d7a7b
Repository checks	421 passed: 104 prompt optimizer, 31 workspace, 66 core, 16 catalog, 172 host, and 32 webview
Type checks	All six package checks passed
Schema check	No unexpected change
Build and package	Passed
Package contents	387 files; 684,130 bytes
Package SHA-256	954ce0102754b79f7b3503bfe0c9053ba30ae21ab95177cc55d28c7643d2567b
Installed local flow	Passed for the recorded same-process journey
Narrow-screen review	360 × 640 review rendered without horizontal overflow
GitHub-hosted exact-version CI	Independently repeated the repository tests, schema check, build, package inspection, installed activation/open, and hygiene checks

Restart evidence boundary

The packaged extension passed the recorded cross-process restart, resume, reset, and project-context preservation journey in an isolated profile. A later repeat was blocked before product checks because macOS automation could not see the consent window. That automation stop did not record a product failure or erase the earlier evidence; independent repetition on the exact current version in a second environment remains useful next-stage evidence.

Phase 4 governance and comparative-evidence boundary

The readiness request was authorized, but the 20,000 aggregate, 120,000 evaluation, and 12,000 per-completion token ceilings were introduced during contract preparation by the AI assistant without explicit approval from the human approver. The response reported 14,531 tokens, and the 12,000 sub-limit stopped the process before any business task, optimized output, or quality comparison. The stop and evidence preservation were correct containment; allowing an unapproved number to gain pivotal authority was a clear governance failure. Prompt quality remains unmeasured.

Appendix B — Key risks and work still to do

Risk or unfinished work	Why it matters	What closes it
Guided answer step is incomplete	DWI can show a question but cannot finish the simple answer journey	Build and test answer, skip, safe use, and old-answer protection
Independent restart repetition is pending	Recorded restart evidence exists, but another environment would strengthen operational confidence	Repeat the full journey on the same exact version during a controlled pilot
Unapproved decision logic entered the contract	A pivotal rule stopped work and shaped usage logic without approval from the human approver	Audit every decision-bearing limit, default, multiplier, exception, and fallback; require trace, approval, version, test, and rollback
Dollar ceilings were not executable controls	Pricing and billing data were unavailable, so actual cost was unknown	Require approved price provenance, pre-call reservation, hard spend enforcement, and post-call reconciliation
Comparative live-AI uplift is not yet measured	The governance failure stopped Phase 4 before any business task or optimized output was evaluated	Resume only after the audit and an explicitly approved accounting contract, then separately authorize the comparison
Magic-number or hidden-multiplier accounting	One unexplained value can control cost, data use, or a phase outcome	Use approved inputs through an imperative policy layer; explain and reconcile every result inside hard limits
Independent provider and operational-pilot evidence is pending	Exact-commit CI proves engineering repeatability, while provider quality and operational adoption answer different questions	Run the authorized provider comparison and then one controlled pilot
Project-management data could become hidden authority	Urgency might improperly affect privacy, safety, or availability	Keep fields minimal, make the link optional, and stop safely on bad data
AI usage allowance could be mistaken for permission	Priority might trigger unauthorized spend or data sharing	Enforce the control order and require explicit external-AI action
Feedback could become employee monitoring	Trust can be lost and activity may be mistaken for productivity	Exclude content and names, use minimum group sizes, and prohibit ranking
Usage estimates may be mistaken for bills	Management decisions could use inaccurate cost information	Label estimates, show uncertainty, and compare with observed use later
Activity counts may become the goal	More usage can hide poor outcomes or repeated attempts	Measure usefulness, correction, time, limits, and safe fallback together

Appendix C — Plain-English glossary

Term	Plain-English meaning
Project Meta Context	The short project summary the developer has reviewed and approved
Proof of concept	A working local demonstration used to test value and controls before wider use
Project-management system	The tool that holds planned tasks, priorities, delivery periods, and work status
Token	A unit used to measure AI input and output; it helps estimate usage, not necessarily the final bill
Adaptive AI allowance	A planned task allowance based on approved factors and kept inside absolute privacy, spending, and call limits; it cannot rise merely because a run used more than expected
Decision provenance	The human approver, approved requirement, reason, calculation, version, test, and rollback behind a pivotal rule
Imperative policy layer	Executable logic that verifies approval, calculates the decision, explains it, and stops before action when authority is missing
Local rule-based draft	A draft prepared by DWI without calling an external AI service
External AI service	The separately configured AI system used only after an explicit disclosed action
Fallback	Continuing safely with the local draft when an external AI action fails or is unavailable
Outcome-only feedback	Simple result information that does not copy prompts, output, code, or project content
Controlled comparison	A fixed test used to check whether the external-AI improvement is genuinely better
Exact-version evidence	A test result that belongs only to the exact product version that produced it

Final boundary: demonstrated current capabilities remain valid, Phase 4 remains frozen after a clear governance failure, and live-AI quality remains unmeasured. Future project-management, adaptive-accounting, and management-feedback features require their own design, explicit approval, implementation, privacy review, and evidence.